

Detección de atacantes humanos vs. ataques automatizados en sesiones SSH

Deep Learning aplicado a la priorización de respuesta a incidentes

CNN · Autoencoder · MLP

Honeypot Cowrie

Wazuh SIEM

Despliegue en AWS

Proyecto Final — Diplomado de Deep Learning

Daniel Pareja Franco - Celeste Duran Junio 2026

Repositorio: github.com/DanielPPPf/Proyecto-Diplomado

Resumen ejecutivo

Las organizaciones reciben **miles de intentos de acceso SSH no autorizados al día**. La inmensa mayoría son automatizados (botnets, escáneres, gusanos) y de bajo riesgo; una fracción mínima corresponde a un **atacante humano interactuando en vivo** ("hands-on-keyboard"), señal de un ataque dirigido y de mayor severidad.

Este proyecto desarrolla y **pone en operación** modelos de deep learning capaces de distinguir ambos casos a partir de las sesiones capturadas por un honeypot, con el fin de **priorizar la respuesta a incidentes** en un SOC. El trabajo abarca desde el análisis exploratorio hasta un **laboratorio funcional en AWS** donde los modelos clasifican ataques reales en tiempo casi real y generan alertas en un dashboard de Wazuh.

213,681

SESIONES ANALIZADAS

0.94

PR-AUC MODELO 1

2

MODELOS DL EN
OPERACIÓN

AWS

LAB EN VIVO VERIFICADO

Resultado principal: el pipeline completo *ataque* → *honeypot* → *inferencia* → *SIEM* → *alerta* funciona end-to-end, clasificando sesiones reales como **AUTOMATED** o **MANUAL** y mostrándolas en el dashboard de Wazuh.

1. Definición de objetivos

Objetivo de negocio

Ayudar al equipo de seguridad (SOC) a **separar el ruido automatizado de las intrusiones humanas activas**, concentrando recursos donde el riesgo real es mayor y reduciendo el tiempo de detección de amenazas dirigidas.

Objetivo técnico

El EDA reveló que las sesiones útiles son escasas y casi todas automatizadas, por lo que el objetivo se formuló como un **enfoque combinado** de dos modelos:

1. **Clasificador de escalada (supervisado)**. Predecir si una sesión será *fuerza bruta* vs. *interactiva* (ejecuta comandos) usando solo señales de la **fase de login**, sin mirar los comandos. Permite una alerta temprana antes de que el atacante actúe.
2. **Detección de anomalías (no supervisado)**. Un **autoencoder** sobre la secuencia de comandos que aprende el payload automatizado "normal" y marca por error de reconstrucción las sesiones **raras** — candidatas a actividad humana o campañas nuevas.

2. Análisis Exploratorio de Datos (EDA)

Dataset: *CyberLab honeynet* (Zenodo 3687527), capturado con el honeypot **Cowrie** en ~50 nodos. Se usó una muestra de **3 días** (24–26 ago 2019): **213,681 sesiones**, ya agrupadas por sesión con sus eventos y marcas de tiempo.

Hallazgo que redefinió el proyecto: el **99.8%** de las sesiones son fuerza bruta pura (la IP prueba credenciales y se desconecta sin ejecutar un solo comando). Solo el **0.2% (429 sesiones)** ejecuta comandos, y de ellas ~91% lo hace a ritmo de máquina con payloads Mirai idénticos. La actividad humana es rarísima → no hay suficientes ejemplos para un clasificador humano-vs-bot directo.

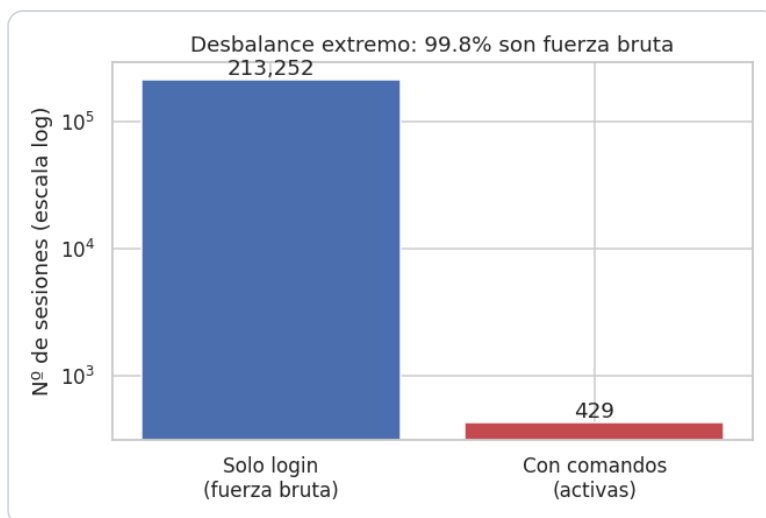


Fig. 1 — Desbalance extremo: 99.8% de las sesiones son fuerza bruta.

Las sesiones activas tienen un **ritmo de comandos delator**: un script ejecuta en milisegundos, mientras un humano hace pausas para pensar. Esta es la señal central del proyecto.

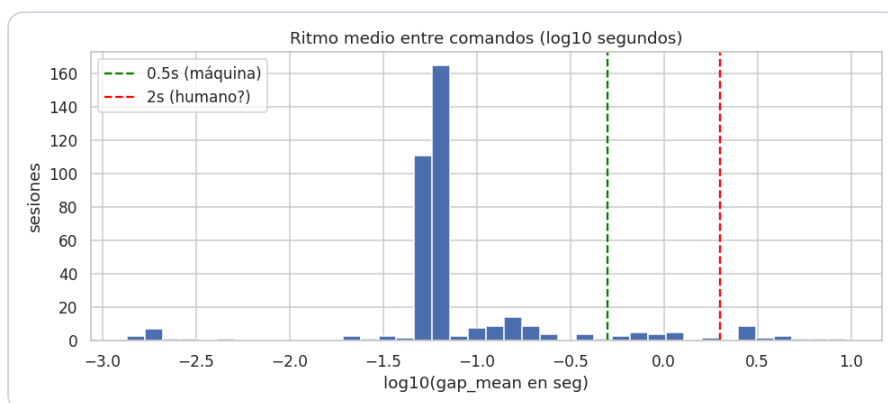


Fig. 2 — Ritmo medio entre comandos ($\log_{10}$ s). El grueso cae por debajo de 0.5 s (ritmo de máquina); muy pocas superan los 2 s (candidatas humanas).

Además, decenas de IPs distintas ejecutan **exactamente la misma secuencia** de comandos (droppers tipo Mirai: `cd /tmp; wget http://.../bins.sh; chmod 777; sh bins.sh`), lo que confirma actividad de botnet.

3. Diseño de la configuración experimental

Aspecto	Decisión
Unidad de análisis	La sesión (un intento de acceso desde una IP)
Parsing	<code>src/parse_cowrie.py</code> agrega eventos → tabla de features por sesión
Particiones (Modelo 1)	70% train / 15% validación / 15% test, estratificado
Validación de honestidad	Split temporal (entrenar días 1-2, probar en día 3)
Métricas	PR-AUC (titular, por el desbalance), ROC-AUC, F1, recall, precisión
Desbalance	<code>class_weight</code> balanceado + umbral ajustado por F1
Entorno	conda <code>diplomadoDL</code> , TensorFlow 2.14 (Keras 2), CPU

4. Modelo 1 — Clasificador de escalada

Tarea: fuerza bruta vs. sesión interactiva, a partir de **49 features** de la fase de login (4 numéricas: duración e intentos de login; 45 one-hot: sensor, país y versión del cliente SSH).
Arquitectura MLP: `Dense(64) → Dropout → Dense(32) → Dropout → sigmoide`.

0.965

PR-AUC (TEST)

1.000

RECALL INTERACTIVA

0.928

PRECISIÓN INTERACTIVA

0.941

PR-AUC SPLIT TEMPORAL

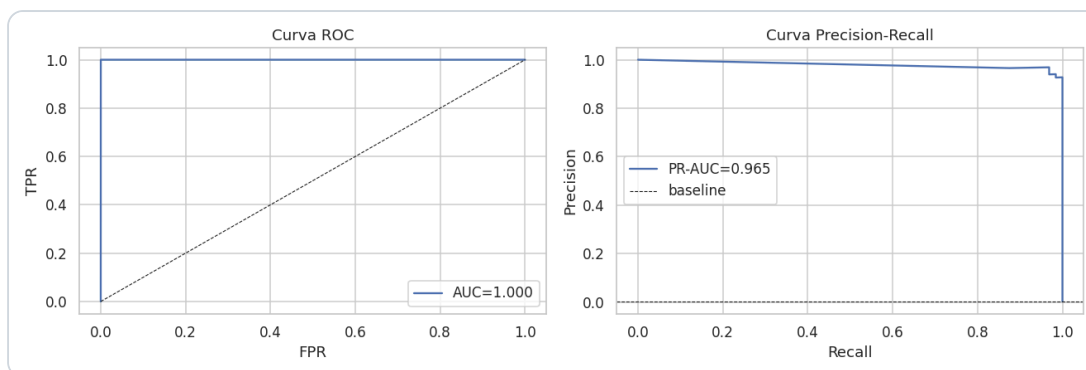


Fig. 3 — Curvas ROC y Precision-Recall del Modelo 1 sobre el conjunto de prueba.

El desempeño **sobrevive al split temporal** (entrenando con dos días y evaluando en un día nuevo se mantiene PR-AUC 0.94), por lo que **no es memorización**: la huella del handshake SSH y el patrón de autenticación predicen genuinamente qué sesiones escalan.

Métrica honesta: reportamos la **PR-AUC (~0.94)** como titular, no la ROC-AUC (que se ve casi perfecta por la abundancia de negativos en un problema con 0.2% de positivos).

5. Modelo 2 — Autoencoder de anomalías

Tarea: sobre las 429 sesiones activas, aprender a reconstruir las secuencias de comandos "normales" (automatizadas). Las que el modelo **no reconstruye bien** son atípicas.

Arquitectura **Conv1D** a nivel de carácter (**L=400**, vocabulario de 89): codificador **Embedding** → **Conv1D+MaxPool** ×2, decodificador **Conv1D+UpSampling** ×2 → **softmax**.

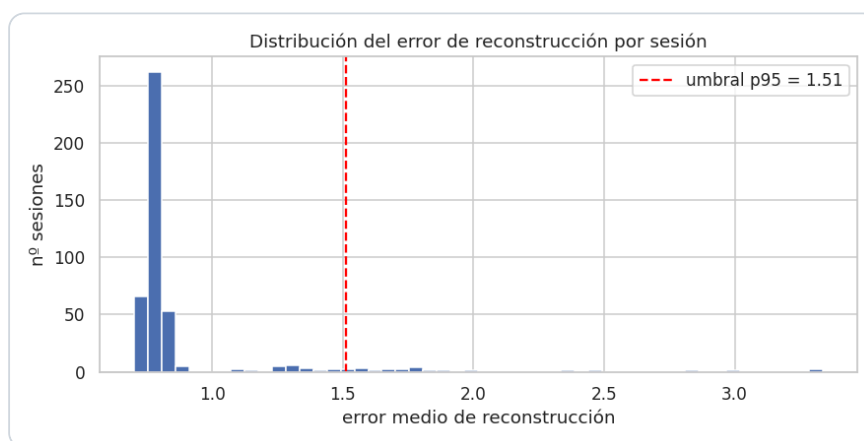


Fig. 4 — Distribución del error de reconstrucción. Umbral p95 ("revisar") y p99 ("campaña novedosa") marcan las sesiones atípicas.

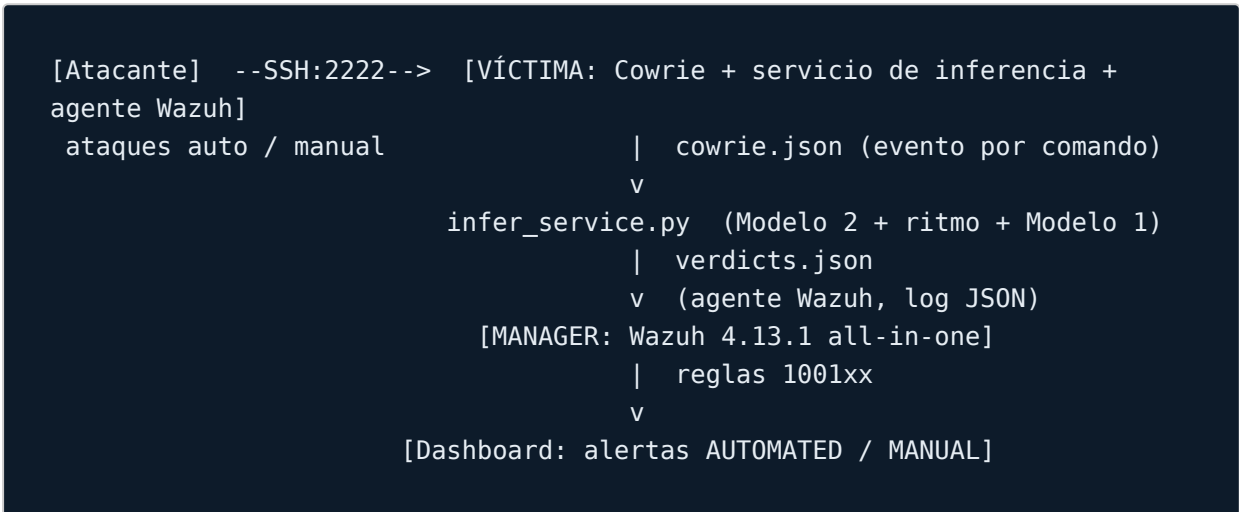
El autoencoder reconstruye con **bajo error los droppers Mirai repetidos** (confirmando que son el patrón dominante) y asigna **alto error a campañas raras** — por ejemplo, una campaña **Pemex.sh** minoritaria, distinta del payload masivo.

Matiz clave: "contenido anómalo" $\neq$ "manual". Un *bot nuevo* también es raro. Por eso el veredicto separa dos ejes: el **ritmo** decide automatizado/manual, y la **anomalía** marca aparte si la campaña es novedosa. La correlación entre ambas señales es baja (0.146): son complementarias.

6. Laboratorio en vivo — de la teoría a la operación

Para demostrar que los modelos **se usan**, no solo se entrenan, se montó un laboratorio en **AWS** donde se atacan honeypots reales y los modelos clasifican en tiempo casi real.

Arquitectura



Componente	Detalle
Infraestructura	2× EC2 m7i-flex.large (8 GB) en us-east-2
Sensor	Honeypot Cowrie en :2222 (mismo esquema de los datos de entrenamiento)
Inferencia	infer_service.py : agrupa sesiones y emite veredicto en verdicts.json
SIEM	Wazuh 4.13.1 + reglas personalizadas (decoder JSON)

Lógica del veredicto en operación

Eje	Señal	Regla Wazuh
AUTOMATED vs MANUAL	Ritmo entre comandos (pausas humanas > 2 s)	100101 (nivel 3) / 100102 (nivel 12)
NOVEDOSA	Error de reconstrucción del Modelo 2	100104 (nivel 10)

Resultados en vivo

Se lanzaron ataques automatizados (comandos en ráfaga) y manuales (tecleados con pausas) contra el honeypot. El servicio clasificó cada sesión correctamente:


```
[AUTOMATED] gap_mean=0.11s recon=1.78 :: ritmo de máquina
[MANUAL] gap_mean=5.23s recon=3.64 :: ritmo humano + campaña
NOVEDOSA
```

El agente reporta al manager como **Active**:

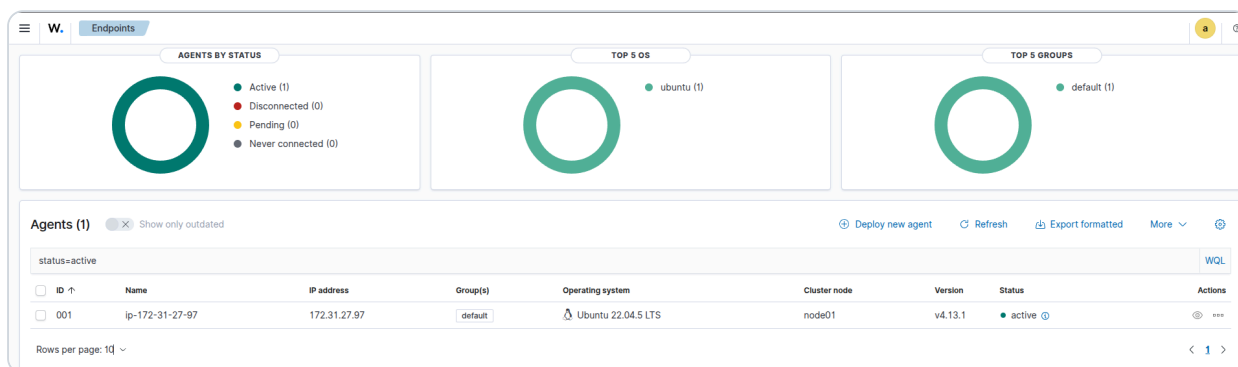


Fig. 5 — Wazuh: el endpoint víctima (ID 001, Wazuh v4.13.1) en estado Active.

Las sesiones aparecen clasificadas en el dashboard, con su regla y nivel de severidad:

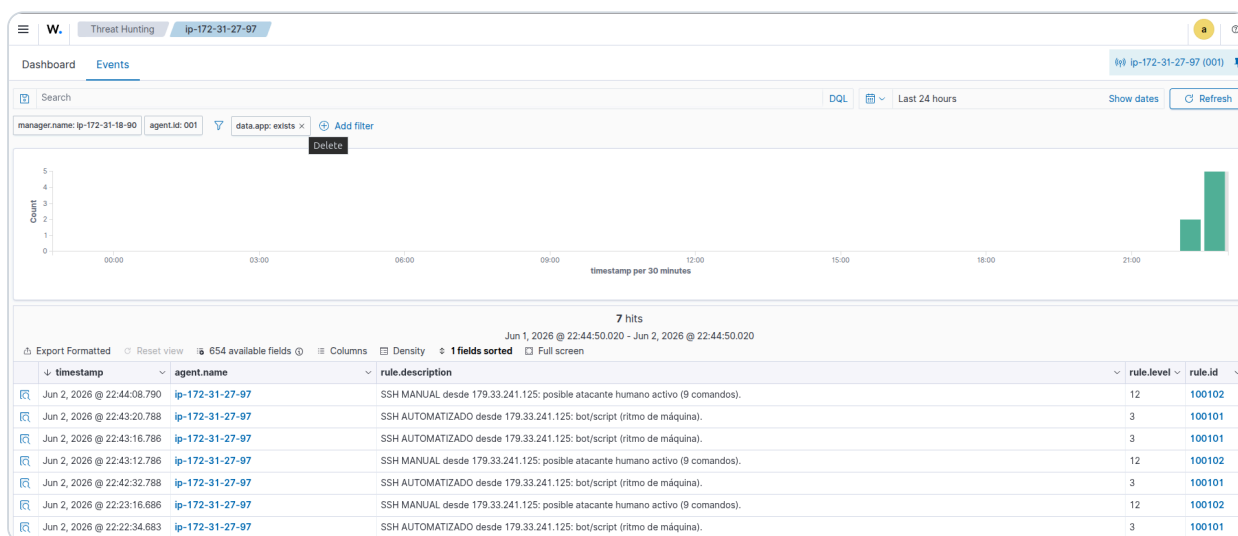


Fig. 6 — Alertas en Wazuh: sesiones SSH MANUAL (nivel 12) y AUTOMATIZADO (nivel 3) con sus reglas 100102 / 100101.

El detalle de cada alerta expone **por qué** el modelo decidió (los campos del veredicto):

a

?

Document Details

View surrounding documents

View single document

×

data.app	ssh-classifier
data.escalation_prob	1
data.gap_mean	5.515000
data.n_commands	9
data.needs_review	true
data.novel_campaign	true
data.reason	ritmo humano (gap_mean=5.52s > 2.0s); campaña NOVEDOSA (recon_err=3.64 > 2.43)
data.recon_err	3.635000
data.rhythm_human	true
data.session	21b86190b294
data.src_ip	179.33.241.125
data.timestamp	Jun 2, 2026 @ 22:44:08.565
data.verdict	MANUAL
decoder.name	json
full_log	{\"app\":\"ssh-classifier\",\"timestamp\":\"2026-06-03T03:44:08.565385Z\",\"session\":\"21b86190b294\",\"src_ip\":\"179.33.241.125\",\"n_commands\":9,\"gap_mean\":5.515,\"recon_err\":3.635,\"novel_campaign\":true,\"rhythm_human\":true,\"needs_review\":true,\"escalation_prob\":1,\"verdict\":\"MANUAL\",\"reason\":\"ritmo humano (gap_mean=5.52s > 2.0s); campaña NOVEDOSA (recon_err=3.64 > 2.43)\"}
id	1700458248.1700053
input.type	log
location	/var/log/ssh-classifier/verdicts.json
manager.name	ip-172-31-18-90
rule.description	SSH MANUAL desde 179.33.241.125: posible atacante humano activo (9 comandos).
rule.firedtimes	3
rule.groups	ssh_classifier, cowrie, ssh_manual, attack
rule.id	100102

Fig. 7 — Alerta **MANUAL**: *gap_mean=5.52s*, *recon_err=3.64*, *novel_campaign=true*, con la razón explicativa.

Document Details		View surrounding documents	View single document
data.app	ssh-classifier		
data.escalation_prob	1		
data.gap_mean	0.210000		
data.n_commands	2		
data.needs_review	true		
data.novel_campaign	false		
data.reason	ritmo de máquina (gap_mean=0.21s); contenido inusual (recon_err=1.78 > 1.51)		
data.recon_err	1.780000		
data.rhythm_human	false		
data.session	bea0aa67554b		
data.src_ip	179.33.241.125		
data.timestamp	Jun 2, 2026 @ 22:43:20.437		
data.verdict	AUTOMATED		
decoder.name	json		
full_log	{"app":"ssh-classifier","timestamp":"2026-06-03T03:43:20.437799Z","session":"bea0aa67554b","src_ip":"179.33.241.125","n_commands":2,"gap_mean":0.21,"recon_err":1.78,"novel_campaign":false,"rhythm_human":false,"needs_review":true,"escalation_prob":1,"verdict":"AUTOMATED","reason":"ritmo de máquina (gap_mean=0.21s); contenido inusual (recon_err=1.78 > 1.51)"} 		
id	1780458200.1707100		
input.type	log		
location	/var/log/ssh-classifier/verdicts.json		
manager.name	ip-172-31-18-90		
rule.description	SSH AUTOMATIZADO desde 179.33.241.125: bot/script (ritmo de máquina).		
rule.firedtimes	4		
rule.groups	ssh_classifier, cowrie, ssh_automated		
rule.id	100101		

Fig. 8 — Alerta **AUTOMATED** (contraste): *gap_mean=0.21s* (ritmo de máquina), regla 100101.

Y la visualización resumen para el SOC:

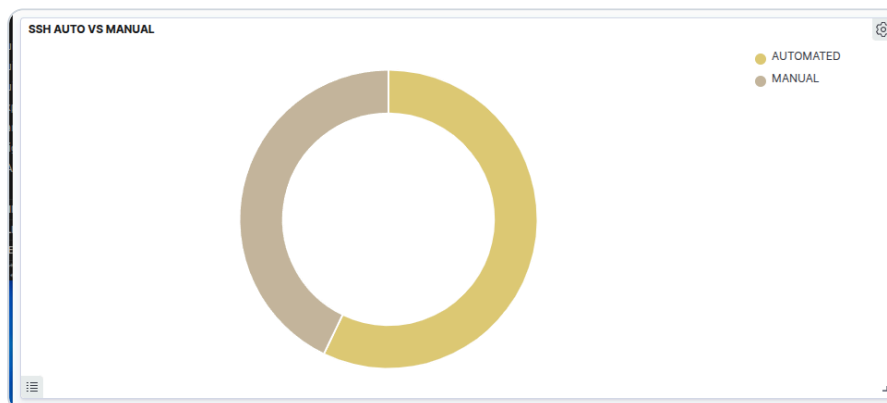


Fig. 9 — Distribución de veredictos **AUTOMATED** vs **MANUAL** sobre los ataques de prueba.

Despliegue y flujo de datos

El modelo se entrena **offline** con datos históricos, pero está diseñado para operar sobre el **flujo diario** de un honeypot vivo: Cowrie escribe eventos continuamente (el honeypot recibió ~72,000 sesiones/día), un job procesa el log y los modelos infieren por sesión. La continuidad

del dato la garantiza exponer el sensor a internet, donde el escaneo automatizado es incesante.

7. Conclusiones

- Se cumplió el objetivo: un sistema de deep learning que **prioriza la respuesta a incidentes** distinguiendo ataques automatizados de actividad humana en SSH.
- El **Modelo 1** predice con alta fidelidad qué sesiones escalarán a ejecutar comandos usando solo la fase de login (PR-AUC 0.94, validado temporalmente).
- El **Modelo 2** aísla las sesiones de comandos atípicas, complementando al ritmo para cubrir casos que una sola señal no detecta.
- El **laboratorio en AWS** demuestra el valor operativo: el pipeline completo funciona end-to-end y las alertas son accionables en el dashboard de Wazuh.

8. Limitaciones y trabajo futuro

- El veredicto AUTO/MANUAL se basa en el **ritmo**: un atacante humano que pegue comandos de golpe se vería como automatizado (lo mitiga parcialmente el Modelo 2).
- Los 3 días de datos son **consecutivos**: comparten campañas de botnet, por lo que no se prueba la generalización a campañas de meses posteriores.
- La actividad humana real es escasa en el honeypot; las etiquetas humano/bot se infieren por heurística, no por verdad de terreno.

Trabajo futuro: ampliar a más meses de captura; incorporar más señales de comportamiento (tipos, exploración del entorno); validar con sesiones humanas etiquetadas; y añadir un modelo de secuencias temporal (LSTM) sobre los tiempos entre comandos.

Referencias

1. *CyberLab honeynet dataset* — Zenodo. <https://zenodo.org/records/3687527>
2. *Towards Identifying Human Actions, Intent, and Severity of APT Attacks Applying Deception Techniques*. <https://arxiv.org/pdf/2006.01849>
3. *GPT-2C: A GPT-2 parser for Cowrie honeypot logs*. <https://arxiv.org/pdf/2109.06595>
4. Cowrie SSH/Telnet Honeypot — <https://github.com/cowrie/cowrie>
5. Wazuh — Open Source Security Platform — <https://wazuh.com>